UNIVERSIDAD TORCUATO DI TELLA

TD7: Ingeniería de Datos

SQL: Parte II

Ordenamiento y paginación

- Para ordenar el resultado y paginar los datos, extendemos el esquema básico de consulta con las clausulas ORDER BY y FETCH FIRST:

```
SELECT A1, A2, …, An
       FROM T1, T2, …, Tn
       [ WHERE condicion ]
[ ORDER BY B1, B2, …, Bm ];
[ OFFSET k ROWS FETCH FIRST n ROWS ONLY ];
```

Ordenamos el resultado primero por B_1; si hay empate por B_1 desempatamos por B_2, etc.

El orden por cada atributo es ascendente, salvo que pongamos la palabra DESC después del atributo

Una vez ordenado, descartamos las primeras k, y luego mostramos las siguientes n

Ordenamiento y paginación: ejercicio

Devolver las 5 facturas con mayor importe, mostrando el identificador de la factura, su importe, y el nombre del usuario a quien se emitió cada factura. Ordenar los resultados en forma descendente por importe.

```
SELECT invoice_id, total, first_Name, last_name
FROM invoice, customer
WHERE invoice.customer_id = customer .customer_id
ORDER BY total DESC
OFFSET 0 ROWS FETCH FIRST 5 ROWS ONLY;
```

Ejercicio integrador 1

Encontrar el nombre y la duración de los 5 tracks con mayor duración de los que se conozca el compositor y que están en la playlist ‘Classical’

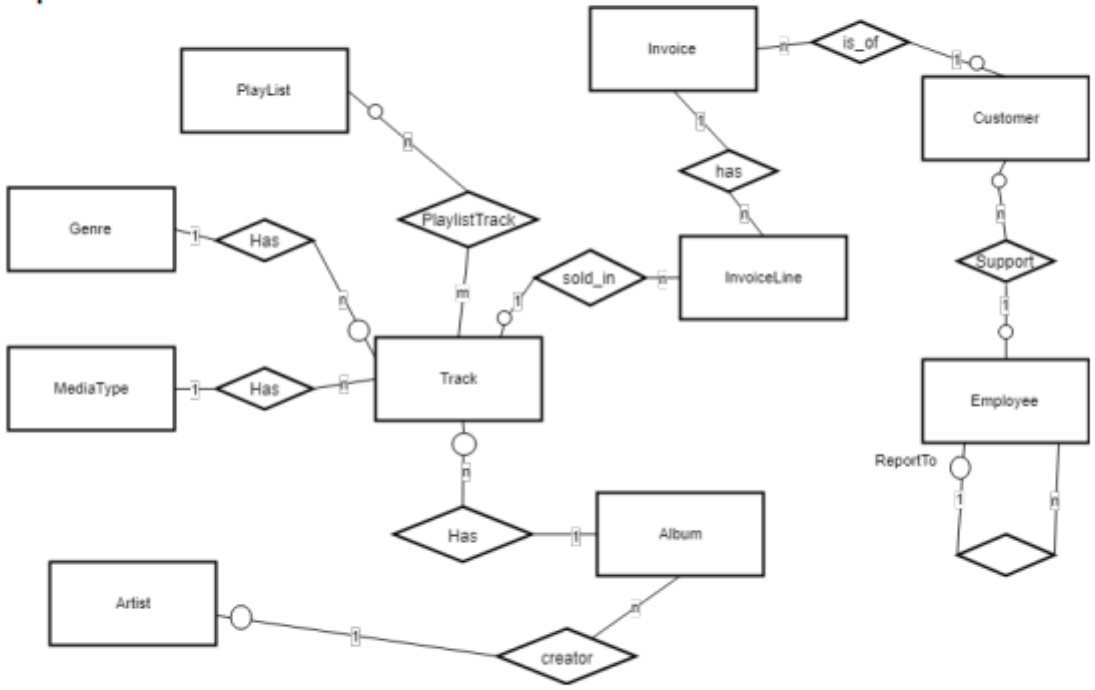
Ejercicio integrador 2

Obtener los nombres de los tracks, sus compositores, el álbum y el artista al que pertenecen de todos aquellos tracks que están en facturas con más de cinco años de emitidas.

Dataset

Base de Datos 'Chinook'

DER simplificado de Chinook:



Agregación en SQL

Agregación en SQL

- Nos gustaría resumir en una tabla la cantidad de títulos ganados por cada tenista y su premio total.
- Para ello necesitamos **agrupar** los datos por cada tenista:

CAMPEÓN

nombre_tenista	nombre_torneo	premio
Novak Djokovic	Abierto de Australia	8.000.000
Novak Djokovic	Abierto de Madrid	2.500.000
Novak Djokovic	Roland Garros	5.000.000
Rafael Nadal	Abierto de Barcelona	1.500.000
Juan Martín del Potro	Abierto de Estocolmo	300.000
Andy Murray	Master de Shanghai	4.000.000
Andy Murray	Master BNP Paribas	2.000.000
Andy Murray	ATP Tour Finals de Londres	4.000.000

Agregación en SQL

- Una vez agrupados, queremos reemplazar cada grupo de datos (que corresponde a un tenista) por una única fila, que muestre el nombre del tenista, pero resumiendo a los torneos ganados indicando su cantidad, y a los premios ganados indicando su suma.

- El resultado sería:

nombre_tenista	cantidad_torneos	premio_total
Novak Djokovic	3	15.500.000
Rafael Nadal	1	1.500.000
Juan Martín del Potro	1	300.000
Andy Murray	3	10.000.000

Agregación en SQL

- La **agregación** en SQL colapsa todas las filas que coinciden en el valor de uno o más atributos (en este caso, en la columna “nombre_tenista”) en una única fila que las representa a todas ellas.

- Decimos que “agrupamos las filas de Campeones por el nombre del tenista, resumiendo los torneos con su cantidad, y los premios con su suma”.

nombre_tenista	cantidad_torneos	premio_total
Novak Djokovic	3	15.500.000
Rafael Nadal	1	1.500.000
Juan Martín del Potro	1	300.000
Andy Murray	3	10.000.000

Agregación en SQL

nombre_tenista	cantidad_torneos	premio_total
Novak Djokovic	3	15.500.000
Rafael Nadal	1	1.500.000
Juan Martín del Potro	1	300.000
Andy Murray	3	10.000.000

- En SQL, esto se logra con la cláusula **GROUP BY**:

Los atributos que NO están en el GROUP BY solo pueden aparecer en el SELECT si pasan por una función de agregación.

Funciones de agregación sobre alguno/s de los atributos restantes

```
SELECT nombre_tenista, COUNT(nombre_torneo), SUM(premio)
FROM Campeon
GROUP BY nombre_tenista
```

Atributo de agrupamiento

Esquema de consulta con agregación en SQL

- El esquema de consulta con agregación en SQL es:

Algunos de los atributos por los que agrupamos a las filas (A_1, A_2, …, A_n) podemos mostrarlos en la "fila resumen" si lo deseamos

```
SELECT Ak1, Ak2, …, Akp, agg(B1), agg(B2), …, agg(Bm)
       FROM T1, T2, …, Tn
       [ WHERE condicion ]
  GROUP BY A1, A2, …, An
  [ HAVING condicion ];
```

En cambio, los atributos que no están en el GROUP BY sólo pueden aparecer en el resultado si pasan por una función de agregación

COUNT(), SUM(), AVG(), MIN(), MAX()

Agregación en el SELECT: ejercicio

1. Encontrar para cada compositor la cantidad de tracks compuestos.

```
SELECT composer, COUNT(trackid)
FROM Track
GROUP BY composer;
```

"composer" no tiene función de agregación porque es el atributo a agrupar.

Esquema de consulta con agregación en SQL

- La cláusula **HAVING** permite filtrar alguno de los grupos, en función del valor de los atributos agregados.

```
SELECT Ak1, Ak2, …, Akp, agg(B1), agg(B2), …, agg(Bm)
       FROM T1, T2, …, Tn
       [ WHERE condicion ]
GROUP BY A1, A2, …, An
[ HAVING condicion ];
```

Aquí podemos poner condiciones sobre los valores agregados de los atributos dentro del grupo.

Esquema de consulta con agregación en SQL: ejercicio

2.Encontrar para cada compositor la cantidad de tracks compuestos **mostrando únicamente aquellos compositores con al menos 20 tracks compuestos**.

```
SELECT composer, COUNT(trackid)
FROM Track
GROUP BY composer
HAVING COUNT(trackid)>=20;
```

Consultas de agregación con más de una tabla en el FROM

- Recordemos que poniendo más de una tabla en el FROM, obtenemos el producto cartesiano entre esas tablas.
- Esto podemos transformarlo en una junta usando la condición correcta en el WHERE, o bien usando el INNER JOIN en el FROM.
- Si además de esto utilizamos un GROUP BY, podremos agrupar y resumir el resultado de esa junta.

Esquema de consulta con agregación en SQL: ejercicio

3.Encontrar quienes son los clientes que tienen facturas con importes promedio más alto. Mostrar a los 10 clientes con mayor importe promedio, mostrando el apellido del cliente, la cantidad de facturas a su nombre y el promedio de esas facturas.

```
SELECT last_name, COUNT(invoice_id), AVG(total)
    FROM customer c INNER JOIN invoice i
    ON c.customer_id = i.customer_id
GROUP BY last_name
ORDER BY AVG(total) DESC
OFFSET 0 ROWS FETCH FIRST 10 ROWS ONLY;
```

Esquema de consulta con agregación en SQL: ejercicio

4.¿Y si quisiéramos saber en cuántas facturas aparece cada track, pero sólo para aquellos que aparecen en al menos dos facturas?

```
SELECT t.name, COUNT(il.invoice_line_id) cantidad
    FROM Track t INNER JOIN invoice_line il
    ON t.track_id = il.track_id
GROUP BY t.name
HAVING COUNT(il.track_id) >= 2
ORDER BY cantidad DESC
```

Ejercicio integrador 3

---

5. Encontrar al máximo fan de Iron Maiden, al que definiremos como aquel que tenga más líneas de factura vinculadas con tracks de la banda. Mostrar el apellido de dicho usuario y la cantidad de líneas de factura vinculadas con tracks de la banda.

```
SELECT c.last_name, count(il.invoice_line_id) cantidad
from invoice_line il   INNER JOIN invoice i on il.invoice_id = i.invoice_id
                       INNER JOIN customer c on i.customer_id = c.customer_id
                       INNER JOIN track t on il.track_id = t.track_id
                       INNER JOIN album a on a.album_id = t.album_id
                       INNER JOIN artist ar on ar.artist_id = a.artist_id
where ar.name = 'Iron Maiden’
group by c.last_name
order by cantidad desc
OFFSET 0 ROWS FETCH FIRST 1 ROWS ONLY;
```

Consultas anidadas

- El resultado de una consulta SQL siempre es una tabla…
- Pero cuando esa tabla tiene solo una fila y una columna, entonces puede ser pensada como un valor…
- y entonces puede ser incluida en cualquier parte en la que SQL espera un valor.

```
SELECT Ak1, Ak2, …, Akp, agg(B1), agg(B2), …, agg(Bm)
       FROM T1, T2, …, Tn
       [ WHERE ( SELECT …
                 FROM …
                 WHERE ) = valor … ];
```

Subconsulta anidada

¡Cuidado! Si el resultado tiene más de una fila, devolverá error

Consultas anidadas: ejercicio

6.Encuentre los nombres de los tracks pertenecientes a facturas del cliente que tiene la factura de mayor importe.

```
select t.name
from invoice i   INNER JOIN invoice_line il on i.invoice_id = il.invoice_id
        INNER JOIN track t on il.track_id = t.track_id
where customer_id = Acá empieza la consulta anidada
        (Select  c.customer_id
        from customer c INNER JOIN invoice i on c.customer_id = i.customer_id
                order by total desc
                OFFSET 0 ROWS FETCH FIRST 1 ROWS ONLY)
```